Question 3

- - / 10

You closed timing in the design flow (had zero violations). When you fabricated the chip it did not work. When you lowered the clock frequency, the chip worked. What violation occured? If more than one is a possibility, choose them.

1. Setup-time violation
2. Hold-time violation
3. Large positive slack
4. Negative slack

Question 4

- - / 10

In the question above (about setup-time violation) what could have caused the violation to occur?

1. You chose a constraint that PAR cannot satisfy
2. Process variations
3. You created a path with a very small delay
4. One of your masks failed to create a supply line

What is the main cause of the deterioration of wire delay as technology scales down

- ○ Capacitance
- ◉ Resistance
- ○ Length
- ○ Width

Which of the following is true about wires where resistance plays a role (choose all that apply)

- [ ] They can never be metal wires
- [x] They may be silicon wires
- [x] Their delay is worse than purely capacitive wires
- [ ] They cause inductive bounce

Decoupling capacitors are introduced to reduce

- ( ) Capacitive loads
- (•) Inductive bounce
- ( ) Resistive drops
- ( ) Clock skew

The higher the metal layer, the: (choose all that may apply)

- [x] Lower the capacitance to substrate
- [x] The farther wires should be spaced
- [ ] The thicker the wire
- [x] The more likely the wire is to travel longer

The following are things that happen as the technology scales down

- [ ] Wire delay becomes less importance
- [ ] Skin effect starts to become less effective
- [ ] Gate delay increases in relative importance
- [ ] Capacitance to the substrate decreases quickly

A3taked ya2ema a5er wa7da ya2ema none lw fih none.

Choose the sentence that best describes skew

- ( ) A stochastic spatial phenomenon related to noise
- ( ) A stochastic spatial phenomenon related to delay
- ( ) A deterministic temporal phenomenon related to delay
- (•) A deterministic spatial phenomenon related to delay

If the clock and data are distributed in opposite directions

- (●) New setup-time violations may emerge, old hold-time violations may disappear
- ( ) New setup-time violations may disappear, old hold-time violations may disappear
- ( ) New setup-time violations may disappear, old hold-time violations may emerge
- ( ) New setup-time violations may emerge, old hold-time violations may emerge

The main advantage of a grid distribution of clock is

- (●) Independent of underlying design
- ( ) Low power
- ( ) Low area
- ( ) Low local skew

Latch based pipelines can be very high performance because

- ( ) We can use alternating active high and active low latches
- ( ) They do not have a setup time
- ( ) Latches have lower propagation delay than registers
- (●) The can borrow slack from each other

The following is a **sufficient** condition to ensure that a latch-based pipeline will not fail

- [ ] Borrowed time in the first stage does not exceed T/2-tsu
- [ ] Total borrowed time in the last stage does not exceed T/2-tsu
- [x] Total borrowed time in every stage does not exceed T/2-tsu
- [ ] Borrowed time in the last stage does not exceed T/2-tsu

If clock 1 and clock 2 have frequencies of 133MHz and 399MHz respectively, you need to do the following when passing data between the two

1. Manage the phase
2. Use synchronizers
3. Use asynchronous FIFOs
4. All or any of the above

Passing data between two clock domains is a problem because

1. It causes violations that repeat very frequently
2. It might cause a setup time violation
3. We might read the same data on multiple cycles
4. We might face a hold-time violation

Why should you not pass a bus through a synchronizer

1. Because we might have a buffer underrun
2. Because we might have a buffer overrun
3. Because there is a one-cycle uncertainty in synchronizers
4. Because the phase of the receiver clock is uncertain

Why can you pass read and write pointers through synchronizers even though they are busses

1. Because they are Grey encoded counters
2. Because they are BCD encoded counters
3. Because they are constant registers
4. Because we only care about their difference

Choose the true statements about an N-bit adder

- [x] Any sum delay is greater than any carry delay
- [x] Carry delay is significantly more important than sum delay
- [ ] Delay will scale linearly with the wordlength of inputs
- [x] The area will increase with the wordlength of inputs

Mesh moqtane3 be awel wa7da la2en lw 4-bit adder masalan fa akeed S_0 el delay beta3ha a2al men C_3.

The critical path of a 20 bit carry bypass adder which has a single block of 20 bits:

- ( ) Passes through the bypass multiplexer
- (•) Passes through the entire carry chain
- ( ) Passes through 4 multiplexers and two carry chains
- ( ) Passes through 2 multiplexers and two carry chains

To find out carry out, you need to know the following:

- [x] Generate
- [ ] Delete
- [x] Propagate
- [x] Carry in

Increasing the number of bits per block in a carry bypass adder

- ( ) Increases the number of blocks
- (•) Increases the ripple carry delay
- ( ) Increases the contribution of multiplexers to delay
- ( ) Decreases delay through the initial block

In a carry bypass adder, sum delay appears in the critical path delay

- ( ) Zero times
- ( ) Once per block
- (•) Only once
- ( ) Once for initial block and once for the final block

What makes RTL different from programming

- ( ) Parallel execution
- (•) Concurrency of statements
- ( ) Pipelining
- ( ) Simulation

What would happen if we forget to write a condition for multiplexing (outside a process)

- ( ) Compilation will pass
- ( ) A warning will be raised
- (•) An error will be raised
- ( ) We will be stopped at placement and routing

Which of the following will count towards total hardware when we synthesize

- ○ An entity declaration
- ○ An architecture
- ◉ An instance
- ○ A component

Which of the following is a good design practice when coding

- [x] Declare all numbers in terms of generics
- [x] Inherit generics from higher levels
- [x] Use for-generate statements for repetitive instantiations
- [x] Use labels for instances

Which of the following is not a valid signal value for a std_logic type

- ○ Z
- ○ W
- ○ H
- ◉ Y

When does a variable assignment turn into an event

- ○ At the end of the process
- ○ At the first wait statement
- ◉ Immediately
- ○ At the beginning of the process

Which of these ports can be shorted together (e.g. clk1 and clk2 connected to the same clk) while the memory is still recognized by the synthesizer as a two port memory.

- [x] clk
- [x] En
- [ ] Add
- [x] WE

Pick any prcoess fragment that produces an unintended latch

- [ ]
```
if (clk'event and clk='1') then
Q <= D;
end if;
```
- [ ]
```
if clk='1' then
Q<=D;
end if;
```
- [x]
```
if sel="00" then
op<=A;
elsif sel="01" then
op<=B;
elsif sel="10" then
op<=C;
elsif sel="11" then
op<=D;
end if;
```
- [x]
```
if sel='0' then
A<=B; C<=D; E<F;
else
A<=K; D<=L; E<M;
end if;
```

Which block cannot be made scalable by using generics?

- [ ] Counters
- [ ] Shift registers
- [ ] Multiplexers
- [ ] Memories

QUESTION 5

Creating an implicit latch will result in synthesis error.

- ( ) True
- (•) False

QUESTION 6

Pick coding practices you do **not** recommend

- [x] Writing an if-else statement without a final unconditional else
- [x] Mixing combinational logic registering in a process
- [x] Underdefining a signal inside an if-else statement to save some code
- [x] Writing code with explicit numbers

QUESTION 1

Which of the following is not synthesizable

- (●) Wait;
- ( ) Wait on clk, reset;
- ( ) Wait on clk;
- ( ) Wait until clk='1';

QUESTION 2

When does a transaction turn into an event

- [x] On any wait statement
- [ ] On any end process;
- [ ] On any begin process
- [ ] On any variable assignment

QUESTION 3

Which state should ALL state machines have

- ( ) Wait
- ( ) Fetch
- ( ) Operate
- (●) Reset

QUESTION 3

Which state should ALL state machines have

- ( ) Wait
- ( ) Fetch
- ( ) Operate
- (●) Reset

QUESTION 4

Which of these cannot be part of a package

- ( ) Function
- ( ) Component
- ( ) Procedure
- ( ) Generic

QUESTION 5

What is true about a function

- (●) It can be called outside a process
- ( ) It can use signal assignment
- ( ) It can return multiple values
- ( ) It has to be pure